



Новый навык за несколько недель

[Войти](#)

MobiArt2

1 час назад

Магазин отдаёт 200 ОК, а денег нет: как проверять приём заказов изнутри WooCommerce

Средний



6 мин



2.8K

WordPress*, PHP*

[Кейс](#)

Проблема, которую не видит ни один uptime-монитор

Классический мониторинг отвечает на вопрос «открылась ли страница». Магазин, у которого сломан приём заказов, отвечает на этот вопрос уверенное «да»: витрина рендерится, товары на месте, корзина складывает суммы, код ответа 200, сертификат валиден, скорость отличная. Не работает только одно — деньги не проходят.

Вот пять способов сломать магазин так, чтобы снаружи он выглядел здоровым:

1. Платёжный шлюз остался в тестовом режиме после работ (Stripe/PayPal/ЮKassa переключили, проверили, вернуть забыли).
2. Обновление плагина или темы сломало страницу оформления — а главная, на которую смотрит монитор, цела.
3. Выросла доля сорвавшихся оплат: заказы создаются, но обрываются на платеже.
4. Сломался серверный конвейер заказа: конфликт плагинов, и заказ либо не создаётся, либо зависает.
5. Поток заказов просто оборвался — а причина может быть любой из четырёх выше.

Общее у всех пяти: **HTTP-проверка снаружи бессильна**. Это, кстати, аргумент не в пользу «давайте гонять headless-браузер по воронке» — к нему вернёмся в конце, — а в пользу того, что часть сигналов живёт только внутри CMS. Мы и пошли внутрь: у нас уже есть плагин-коннектор для WordPress, который раз в час собирает диагностику и отправляет её наружу. Задача — научить его понимать, принимает ли магазин заказы.

Три дешёвых сигнала из статистики заказов

Первое искушение — «алертить, если заказов нет». Первое же следствие — алерты у магазина, который делает два заказа в неделю. Поэтому все пороги мы нормируем на **собственную норму конкретного магазина**, а не на абсолютные числа.

Всплеск неудачных заказов. Сравниваем долю `failed` за сутки с семидневной нормой самого магазина:

```
$failed_24h = self::commerce_order_count('wc-failed', time() - DAY_IN_SECONDS);
$total_24h  = self::commerce_order_count(null, time() - DAY_IN_SECONDS);
$failed_7d  = self::commerce_order_count('wc-failed', time() - 7 * DAY_IN_SECONDS);
$total_7d   = self::commerce_order_count(null, time() - 7 * DAY_IN_SECONDS);

if ($failed_24h >= 3) {
    $share_24h = $total_24h > 0 ? $failed_24h / $total_24h : 1;
    $share_7d  = $total_7d > 0 ? $failed_7d / $total_7d : 0;
    if ($share_24h > 3 * $share_7d) {
        // critical: «N неудачных из M за сутки, 7-дневная норма ~X%»
    }
}
```

Объяснить с  SourceCraft

Два предохранителя от ложных срабатываний: абсолютный минимум (`>= 3` — на трёх заказах в сутки статистики нет) и относительный (втрое выше собственной нормы). Магазин, у которого часть заказов всегда срывается — например, с оплатой при получении, — живёт со своей нормой и не «горит».

Засуха заказов. Ноль заказов за сутки — сигнал, только если магазин обычно их делает:

```
if ($total_24h === 0) {
    $avg_14d = self::commerce_order_count(null, time() - 14 * DAY_IN_SECONDS) / 14;
    if ($avg_14d >= 3) {
        // warning: «обычно ~N заказов/день, за сутки — ни одного»
    }
}
```

Инженерная деталь: считать заказы через `wc_get_orders(['paginate' => true, 'limit' => 1, 'return' => 'ids'])` и брать `total` — WooCommerce отдаёт `found_rows`, не таща в память список ID. На магазине с сотней тысяч заказов разница между «взять число» и «взять массив» — это разница между работающим cron и упавшим по памяти.

Платёжный шлюз в тестовом режиме. Единственная проверка, где пришлось спуститься к специфике провайдеров: универсального «дай мне режим шлюза» в WooCommerce нет. Идём по включённым шлюзам (`WC()->payment_gateways()->get_available_payment_gateways()`), и для известного списка боевых провайдеров смотрим стандартные ключи настроек (`testmode`, `sandbox`). Важное самоограничение: **allowlist известных ID, а не поиск по всем настройкам подряд**. Иначе первый же плагин с опцией `sandbox` в своих настройках даст ложный `critical`. И наружу уходит только вердикт и ID шлюза — значения ключей не покидают сайт.

Smoke-прогон конвейера заказа

Статистика ловит поломку постфактум: сначала сорвалось несколько заказов, потом мы это заметили. Хочется активной проверки — но такой, которая не оставит следов в живом магазине.

Раз в сутки плагин прогоняет служебный заказ через настоящий WooCommerce API: создать скрытый виртуальный товар → создать заказ → добавить товар → посчитать суммы → провести по статусам → удалить всё за собой.

```
$step = 'create_product';
$product = new WC_Product_Simple();
$product->set_name('Pingvera smoke test');
$product->set_catalog_visibility('hidden'); // не в каталоге и не в поиске
$product->set_virtual(true); // без доставки и остатков
$product->set_manage_stock(false);
$product->set_regular_price('1');
$product_id = $product->save();

$step = 'create_order';
$order = wc_create_order();

$step = 'add_product';
$order->add_product($product, 1);

$step = 'calculate_totals';
```

```
$order->calculate_totals();

$step = 'set_pending';
$order->update_status('pending');

$step = 'set_cancelled';
$order->update_status('cancelled');
```

Объяснить с 

Переменная `$step` — не украшение. Если конвейер падает, ценность не в самом факте падения, а в том, **на каком шаге** он упал: «не сохранился товар» и «не считаются суммы» — это разные диагнозы для разработчика, который придёт чинить. В `catch` мы отдаём шаг вместе с сообщением:

```
} catch (\Throwable $e) {
    return ['status' => 'fail', 'step' => $step, 'ts' => time(), 'message' => $e->getMessage()];
} finally {
    // что бы ни случилось — прибраться за собой
    self::cleanup_order(self::$pending_order_id);
    self::cleanup_product(self::$pending_product_id);
    self::restore_emails($filters);
}
```

Объяснить с 

Успех не порождает никакой находки вообще: **отсутствие сигнала и есть «здоров»**. Это сознательный выбор в пользу exception-first — модель диагностики не должна плодить зелёные записи, которые никто не читает.

Грабля №1: письма WooCommerce не глушатся тем способом, который приходит в голову первым

Служебный заказ проходит по статусам, а WooCommerce на смену статуса шлёт письма — админу магазина и «клиенту». Прогонять такое ежедневно на живом магазине нельзя: каждое утро владелец получал бы «Новый заказ» и «Заказ отменён» от несуществующего покупателя.

Очевидное решение — фильтр `woocommerce_email_classes`: вернуть пустой массив, и писем не будет. Не работает. `WC_Emails::init_transactional_emails()` навешивает хуки на события заказа ещё на `init` — то есть **к моменту запуска нашего cron-задания хуки уже развешаны**, и подмена списка классов постфактум ничего не меняет: обработчики уже сидят на своих событиях.

Бить нужно в точку, которую WooCommerce проверяет **непосредственно перед отправкой**

— `WC_Email::is_enabled()` , а он дёргает фильтр

`woocommerce_email_enabled_{id}` :

```
foreach (WC()->mailer()->get_emails() as $id => $email) {  
    $hook = 'woocommerce_email_enabled_' . $id;  
    add_filter($hook, '__return_false', 9999);  
    $hooks[] = $hook;  
}  
add_filter('pre_wp_mail', '__return_true', 9999); // страховка уровня ядра WP
```

Объяснить с 

Второй фильтр — `pre_wp_mail` (WP ≥ 5.7) — это подстраховка на уровне ядра: если письмо попытается отправить что-то помимо WooCommerce (плагин уведомлений, интеграция с CRM), оно всё равно не уйдёт. Оба фильтра снимаются в `finally` , чтобы прогон не «проглотил» настоящие письма настоящего заказа, случившегося в ту же секунду.

Грабля №2: `finally` не спасает от фатальной ошибки

`try/finally` гарантирует уборку при исключении. Но PHP-фатал (исчерпание памяти, ошибка в чужом плагине, повешенном на `woocommerce_order_status_changed`) — это не исключение, и `finally` не выполнится. Итог: в магазине останется висеть служебный товар и брошенный заказ. Тихая порча чужих данных — худшее, что может сделать диагностический инструмент.

Поэтому поверх `finally` — `register_shutdown_function` , которая доубирает то, что успели создать:

```
private static $pending_order_id = 0;  
private static $pending_product_id = 0;  
  
public static function shutdown_cleanup() {  
    self::cleanup_order(self::$pending_order_id);  
    self::cleanup_product(self::$pending_product_id);  
}
```

Объяснить с 

ID пишутся в статические поля **сразу после создания сущности**, а не по завершении шага — иначе фатал между «создали» и «записали ID» оставит сироту, о которой мы не узнаем. И сам `cleanup` обёрнут в свой `try/catch` : ошибка уборки не должна перекрыть

исходную причину падения, иначе в отчёте вместо «не считаются суммы» окажется «не удалось удалить заказ».

Почему не headless-браузер

Идея «прогонять реальную воронку в Chrome» напрашивается, и в конце концов мы к ней придём — но как к отдельному уровню, а не как к замене этому.

Внутренний слой стоит одного PHP-процесса раз в час, видит серверную часть заказа и статистику оплат — и не видит того, что живёт только в браузере покупателя: упавший JS корзины, конфликт скриптов на витрине, не загрузившийся платёжный iframe, сломанную заэкшированную страницу из CDN. Браузерный синтетик видит ровно это — и стоит на порядок дороже: браузер на пробере, хрупкие сценарии под каждый магазин, тестовые заказы в чужой базе.

Плюс у внутреннего слоя есть честная слепая зона: **если сайт лежит, изнутри никто не пожалуется** — некому. Это закрывает внешний мониторинг, который у нас и так есть. Два слоя прикрывают слепые зоны друг друга, и это, кажется, единственный способ отвечать на вопрос «магазин работает?» не враньём.

Что получилось

Пять проверок (поток заказов, доля сорвавшихся оплат, тест-режим шлюза, доступность страницы оформления, суточный smoke конвейера), которые живут внутри WordPress-плагина, ходят по расписанию его собственного cron и шлют наружу только вердикты. Критичные становятся событиями мониторинга с алертом, предупреждения — попадают в отчёт.

Честная оговорка напоследок: фича свежая, на боевых магазинах живого пробега пока нет — только тестовые. Так что если у вас есть WooCommerce-магазин и желание проверить эти пороги на реальной статистике, будем рады забрать вашу критику. Особенно интересуют магазины с высокой нормальной долей неоплаченных заказов — именно они первыми покажут, достаточно ли консервативна наша нормировка.

Мораль, если убрать WooCommerce из скобок: проверять надо не то, что легко проверить (код ответа), а то, ради чего система существует (заказ доходит до оплаты). Это почти всегда дороже и почти всегда — единственное, что имеет смысл.

Теги: WooCommerce, WordPress, мониторинг, PHP, e-commerce, платежи

Хабы: WordPress, PHP

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



3

Карма

@MobiArt2

Пользователь

Подписаться



 Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



monobogdan

22 часа назад

Полный разбор схемотехники кнопочного телефона. Samsung C100 — маленькое чудо корейской инженерной мысли



Простой



11 мин



15K

Ретроспектива



+46



26



43



runaway_llm

16 часов назад

Слишком просто, чтобы быть правдой? GPT-5.6 выдал доказательство 50-летней математической гипотезы



Простой



4 мин



13K

Обзор



+37



12



7



YuriPanchul

21 час назад

Из обучения школьников цифре на макетке нужно выкинуть все резисторы и вставить микросхемы с FIFO



Простой



3 мин



16K



+31



41



138



oggr

6 часов назад

Кризис найма (как в IT) журналисты прошли ещё в 2013. Как у них получилось выйти?

🕒 9 мин 👁 7.6K

Мнение

📦 +29

🔖 13

💬 27



Bright_Translate

3 часа назад

Басня о полусыром продукте

🟢 Простой 🕒 11 мин 👁 4.9K

Кейс

Перевод

📦 +24

🔖 4

💬 1



Lunathecatt

23 часа назад

Лёгкий кастомный стратокастер из дешёвых комплектующих

🟢 Простой 🕒 8 мин 👁 11K

Тutorial

📦 +24

🔖 4

💬 4



vital_pavlenko

1 час назад

Как я строил свой продукт, не увольняясь из найма. И что из этого вышло

🕒 6 мин 👁 3.4K

Ретроспектива

📦 +16

🔖 2

💬 7



Velibekov

16 часов назад

Как Яндекс.Навигатор может вести водителей в никуда или в туда, но не туда

🟢 Простой 🕒 6 мин 👁 15K

Кейс

📦 +15

🔖 10

💬 38



Geronom
19 часов назад

Бенчмаркая foreach и эnumераторы: аллокации, которые прятались 10 лет



Простой



17 мин



11К



+15



10



3



Kova13v
23 часа назад

Я запретил Claude говорить «всё работает» без пруфов. Это изменило всё



Средний



9 мин



11К

Кейс

Из песочницы



+15



32



21

Программируем дома: пишем о DIY-проектах, где пригодился код, и получаем призы

Турбо

Показать еще

ВОПРОСЫ И ОТВЕТЫ

Как настроить время сессии в WordPress для редактирования страницы?

WordPress · Средний · 1 ответ

Как задать цвет шрифта строчки меню, соответствующий той странице, на которой пользователь сайта находится в данный момент?

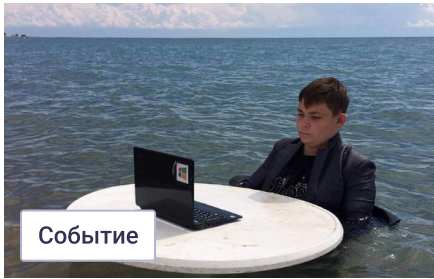
WordPress · Средний · 2 ответа

Как сделать что бы переменная `avatar($ank['id'])`; не конфликтовала с другим файлом?

PHP · Средний · 2 ответа

Больше вопросов на Хабр Q&A

МИНУТОЧКУ ВНИМАНИЯ



Лето по расписанию: календарь событий Хабра



Кто твой супергерой в IT? Исследуем работодателей



Отпуск? Обучение? Курсы? Есть скидки на все

БЛИЖАЙШИЕ СОБЫТИЯ



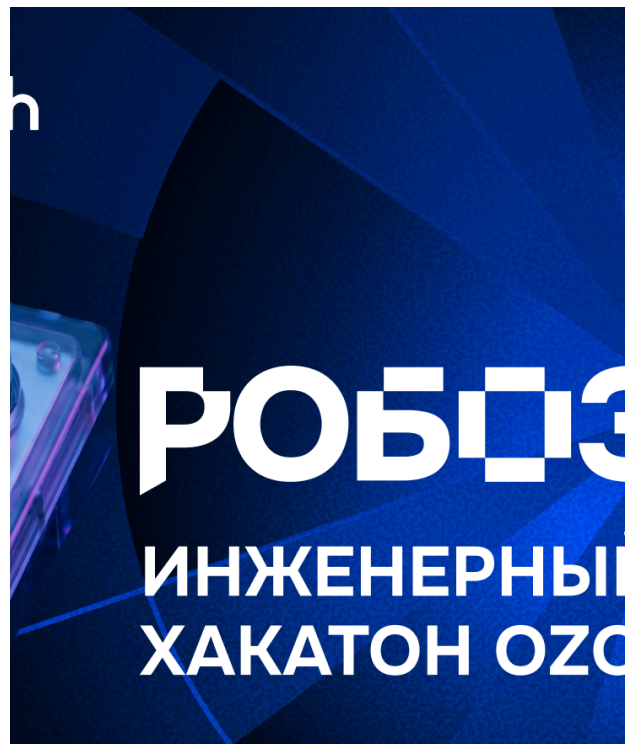
2 марта – 10 августа

PROBA Awards — международная коммуникационная премия, учрежденная в 2000 году

Санкт-Петербург

Маркетинг

Больше событий в календаре



11 июня – 2 сентября

Робозон: хакатон с призовым фондом 15 млн руб.

Москва • Онлайн

Разработка

Больше событий в календаре



15 июля

Веби эпоха

Онлайн

Маркетинг

Больше событий в календаре



 [Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr